

House Rules

ar016 · 20 June 2026

The conventions every contributor (human or agent) must follow when working in this repository. These were previously kept in *CLAUDE.md* at the repo root; that file now points here. Read this before making any edits, alongside the reference articles (parameters, metrics, training, the pinglab-cli) and the project-memory *feedback_notebook* entries, which together hold the layout, glossary, invariants, and conventions every edit must respect. (The former *src/docs/src/pages/styleguide.md* was migrated into those articles and memory and no longer exists as a single file.)

Tooling

- Use *uv* for Python and *bun* for JavaScript.
- Read all the READMEs in the repo before working.
- There are no side effects — running simulations and tests locally is safe.
- Run things in the background where possible.

Running experiments

- When making runs, always report timing: start time, current time, and ETA.
- On Modal, also report the estimated cost.

Modal spends real money

- Do **not** dispatch jobs to Modal (the *-modal-gpu* flag) without explicit permission. Default to local runs; only use *-modal-gpu* when told to.

Version control

- Do **not** create branches or open pull requests unless explicitly asked — commit to the current branch (usually *main*) and stop. “Commit and push” means commit + push, not branch + PR.
- The author drives version control. Do not repeatedly offer to commit; end a piece of work with the result and a *science-focused* next step, not repo bookkeeping. Still commit or push when explicitly asked.

GitHub

- **Never** comment on GitHub issues — no posting or replying. Reading issues is fine; writing to them is not.

House notebook rules

- Notebook runners (*src/notebooks/nbNNN.py*) take only *-modal-gpu* (dispatch target) plus any lifecycle flags they support (*-no-wipe-dir*, *-skip-training*). The old *-tier* size knob is **retired** — each runner hardcodes its own run scale.
- Every hyperparameter — learning rate, epochs, samples, readout, surrogate slope, regulariser strengths, and so on — is hardcoded as a literal in the runner. The runner declares its run scale in a *SCALE* dict passed to *run_dirs.prepare*, which stamps it into the run manifest; the entry renders that scale as a Methods table via the *RunScale* component (single source of truth — the prose never restates the numbers). A training notebook must declare *SCALE* and carry a *RunScale* block in its Methods section.

- New scientific knobs go on the `pinglab-cli` (`src/cli/cli.py`) as flags; the notebook just passes the recipe value inline. The notebook *is* the recipe — reproducing a result must not require remembering flags.
- **Provenance is captured, not assumed.** `run_dirs.prepare` stamps a `__manifest.json` every run (id, UTC time, commit + dirty flag, host, scale). A run with uncommitted dependency code also captures a `__dirty.patch`, so even an uncommitted run is reproducible (checkout the commit, apply the patch, re-run) — you do **not** need to commit before running. The per-entry status bar reads this to show last-run date, the locked commit, and staleness.
- **Train once, reuse many (gamma-gated-sparsity collection).** The canonical networks are trained in one place — nb022 (Training) — to a shared artifact root, and analysis notebooks load those cells via `nb022.load_cell` instead of retraining their own. This deliberately overrides the older “standalone runner, no cross-notebook helpers” convention for this collection: the redundant baseline was being retrained five times over, and the probe notebooks had drifted to lighter epoch budgets. The standard is 100 epochs at $dt = 0.1$ ms (nb044’s dt sweep is the documented exception). A migrated notebook keeps only its analysis; its training block is gone.

Write-up conventions for a *paper*-structured entry:

- **Structure: Abstract → Methods → Results.** Methods is a numbered (itemised) list of the steps. Results is **figures only** — no prose body; the findings live in the captions.
- **Abstract shape: hypothesis → experiments → verdict.** Short and high-level: state the hypothesis clearly first, then a one-line description of the experiment(s) run, then a sentence on whether it holds. No long motivation.
- **Captions are standalone, for a cold read.** Each caption names the axes/panels, the conditions, defines its terms inline (SNR, M, raster, and so on), and ends with the takeaway — so a reader who jumps straight to a plot understands it without the body text.
- **No number prefix in the entry title.** The id lives in the status badge (id · status · date · collection); the title is the bare sentence (for example “Gamma band tightness is a precision signal a neuron can read”, not “055 — ...”).
- **Report results honestly, including partial or negative ones,** folding the caveats into the captions rather than hiding them; prefer one coherent entry over several linked stubs.

House figure rules

- **Plots are black by default — keep them black as much as possible.** Use ink black (`theme.INK_BLACK`, `#1a1a1a`) for a trace; introduce a second colour, signal red (`theme.DEEP_RED`, `#c8102e`), only when two or more traces share one axis and must be told apart. **Separate subplots stay black** — do not colour panels differently for decoration or just because they show different conditions. Fixed exception: an excitatory (E) population trace is **always black** and an inhibitory (I) population trace is **always red**. Reach for the theme accents (amber, electric cyan) or the grey ramp only when a third-or-later overlaid series strictly needs it — never an ad-hoc hex.
- Figures are **strictly 16:9** — no exceptions, including reliability diagrams and anything you would be tempted to make square. Use a figsize whose width:height is 16:9 (e.g. 8×4.5 , 10×5.625 , 12×6.75), with wider widths for more side-by-side panels; never a square or portrait frame.
- The matplotlib theme saves with a tight bounding box, which trims whitespace and skews the saved ratio. Set `savefig.bbox` to `standard` in the plot (after applying the theme) so the

saved canvas is exactly $\text{figsize} \times \text{dpi}$, i.e. exactly 16:9. Verify the saved pixel dimensions if in doubt.

- No log axes unless instructed.
- In matplotlib titles, labels, and annotations, use the Unicode $\approx$ for “approximately” — never an ASCII tilde.

Prose style (docs)

- Never use a tilde or `\sim` for “approximately”. Use $\approx$ in prose and `\approx` inside math. `\sim` is reserved for “distributed as” (for example, $u \sim \text{Uniform}(0, 1)$).
- No backtick inline code in `src/docs/` markdown — use plain text, italics, or math instead (this article included).